Module 05 — Lists

Phase: Core Python
Previous: Strings
Next: Tuples and Sets

Module Outcome

Create, update, grow, and shrink lists, loop them, nest one level, and keep a real copy instead of an alias.

By the end of this module you will be able to:

- Create a list and read with index, slice, and `len`
- Replace an item; add with `append` / `extend` / `insert`
- Delete with `remove`, `pop`, and `clear`
- Use `count`, `index`, and `sort` vs `sorted`
- Loop a list and pair two lists with `zip`
- Read and lightly update a nested list (rows)
- Explain `b = a` vs `a.copy()` — this is a core trap

1. Why This Concept Matters

A string is one piece of text. A class register, a cart, or a queue of tickets is **many** values, in order, that must change: add a student, fix a mark, drop a cancelled order.

```
cart:  Notebook → Pen → USB Cable
marks: 88, 76, 91
```

That bag is a **list**. Order matters. Duplicates are allowed. You can edit it later — unlike a string.

2. Create and read

Understand

A list is written with square brackets. Index `0` is first; `-1` is last. Slice stop is **exclusive** — same rule as strings and `range`.

Store several values under one name and peek by position.

How It Works

```
products = ["Notebook", "Pen", "USB Cable"]
expenses = [450, 1200, 89, 2500]

print(products[0])
print(products[-1])
print(expenses[1:3])
print(len(products))
```

Expected Output:

```
Notebook
USB Cable
[1200, 89]
3
```

⚠️ WATCH OUT

```
# Wrong — index past the end
cart[10]              # IndexError when the list is shorter

# Prefer
len(cart)             # know the length first
if cart:              # non-empty before you read [0]
```

Try It

Store three canteen items and print the first, the last, and the length.

---

3. Update and add

Understand

Lists are **mutable** — assign to an index to replace. Methods grow the list in place.

Action	Method / syntax
Replace one item	`lst[i] = new_value`
Add one at the end	`append(x)`
Add many, flat	`extend([a, b])`
Add at a position	`insert(i, x)`

Fix a misspelled name, add a promo line, grow a cart — without building a brand-new list every time.

How It Works

```
cart = []
cart.append("Notebook")
cart.extend(["Pen", "USB"])
cart.insert(0, "PROMO-Mug")
print(cart)
```

Expected Output:

```
['PROMO-Mug', 'Notebook', 'Pen', 'USB']
```

⚠ WATCH OUT

```
# Wrong — nests a list inside the cart
cart.append(["Pen", "USB"])

# Prefer — one flat add
cart.extend(["Pen", "USB"])
```

Try It

Start with `names = ["Ananya", "Ravi"]`. Change index `1` to `"Ravi Kumar"`, then `insert(0, "Mentor")`. Print the list.

---

4. Delete items

Understand

Remove a line you no longer need — a cancelled order, or a student who left.

Call	Effect
`remove(x)`	Delete the **first** matching value
`pop(i)`	Delete by index; **return** the item (`pop()` = last)
`clear()`	Empty the whole list

Use `pop` when you still need the removed value. Use `remove` when you know the value, not the index.

How It Works

```
orders = ["ORD-101", "ORD-102", "ORD-103"]
cancelled = orders.pop(1)
print(cancelled)
print(orders)
```

Expected Output:

```
ORD-102
['ORD-101', 'ORD-103']
```

⚠ WATCH OUT

```
# Wrong — value not in the list
orders.remove("ORD-999")    # ValueError

# Prefer
if "ORD-999" in orders:
    orders.remove("ORD-999")
```

`pop(i)` raises `IndexError` if the index is missing. `pop()` on an empty list does the same.

Try It

From `["Login", "API", "Deploy", "Docs"]`, `pop()` the last task, then `remove("API")`. Print the cancelled name and what remains.

---

5. Search and order

Understand

Call	Job
`count(x)`	How many times `x` appears
`index(x)`	First index of `x` (`ValueError` if missing)
`sort()`	Sort **in place** (returns `None`)
`reverse()`	Reverse **in place**
`sorted(lst)`	**New** sorted list; original unchanged

Count duplicates, find a score, rank marks — without writing a search loop every time.

How It Works

```
marks = [78, 82, 91, 65, 88, 82]
print(marks.count(82))
print(marks.index(91))

scores = [88, 72, 95]
ranked = sorted(scores, reverse=True)
print(ranked)
print(scores)
```

Expected Output:

```
2
2
[95, 88, 72]
[88, 72, 95]
```

⚠ WATCH OUT

```
# Wrong — sort() returns None
top = marks.sort()    # top is None; marks was mutated

# Prefer
marks.sort()                          # mutate in place
top = sorted(marks, reverse=True)     # new list
```

Try It

Given `[67, 91, 88]`, make a descending sorted **copy** with `sorted`. Print the copy, then the original, to prove the original is unchanged.

6. Loop and `zip`

Understand

`for item in lst:` walks values. Use `range(len(lst))` when you need the index.

`zip(a, b)` walks two parallel lists together — names with marks, products with prices.

If each item is itself a small row of two or three pieces, you can unpack: `for name, mark in rows:`.

How It Works

```
for amount in [200, 1500, 75]:
    if amount > 500:
        print(f"₹{amount}")

for name, mark in zip(["Priya", "Ravi"], [88, 76]):
    print(f"{name}: {mark}%")
```

Expected Output:

```
₹1500
Priya: 88%
Ravi: 76%
```

⚠ WATCH OUT

```
# Wrong — remove while looping the same list
for x in lst:
    lst.remove(x)     # items can be skipped

# Prefer
build a new cleaned list, or loop a copy:  for x in lst[:]
```

`zip` stops at the **shorter** list. Extra items on the longer side are ignored.

Try It

Print amounts above ₹100 from `[120, 80, 450, 60]` using a `for` loop (not a one-liner).

---

7. Nested lists

Understand

A list of lists is a table: outer index = row, inner = column. Read with `rows[row][col]`. Update one cell: `rows[1][1] = 94`.

A marks sheet or an order of `[name, qty, price]` lines is this shape.

How It Works

```
rows = [["Priya", 78, 82], ["Ravi", 91, 70]]
print(rows[1][2])
```

```
order = [["Chai", 2, 20], ["Samosa", 3, 25]]
total = 0
for name, qty, price in order:
    line = qty * price
    total += line
    print(f"{name}: ₹{line}")
print(f"Total: ₹{total}")
```

Expected Output:

```
70
Chai: ₹40
Samosa: ₹75
Total: ₹115
```

⚠️ WATCH OUT

```
# Wrong — row does not exist
rows[2][1]            # IndexError when there are only two rows
```

Indices are 0-based. Check `len(rows)` when you are unsure.

Try It

From `[["Tea", 2, 15], ["Samosa", 1, 25]]`, print each line total and the grand total.

---

8. Alias vs copy

Understand

`b = a` does **not** copy. Both names point at the **same** list. Editing `b` changes `a`.

`a.copy()`, `list(a)`, or `a[:]` make a **shallow** copy — a separate outer list. Nested inner lists may still be shared.

A “backup” cart must not silently rewrite the live cart.

```
a ──┐
    ├──► [1, 2, 3]     one list, two names
b ──┘

a.copy() → new list [1, 2, 3]
```

How It Works

```
a = [1, 2]
b = a
b.append(3)
print("alias:", a)

a = [1, 2]
b = a.copy()
b.append(3)
print("copy a:", a)
print("copy b:", b)
```

Expected Output:

```
alias: [1, 2, 3]
copy a: [1, 2]
copy b: [1, 2, 3]
```

Flat copies are independent. Nested rows are not:

```
original = [["Priya", 88], ["Ravi", 76]]
snapshot = original.copy()
snapshot[1][1] = 91
print(original[1][1])
```

Expected Output:

```
91
```

`.copy()` is enough for a **flat** backup (`[88, 76, 91]`). Nested tables share inner rows unless you copy each row (or deep-copy — interview, not required for the lab).

⚠️ WATCH OUT

```
# Wrong — thinking assignment is a backup
backup = cart
backup.append("Mug")   # cart changed too

# Prefer
backup = cart.copy()
```

Try It

Let `original = [67, 91, 88]`. Set `b = original`, then `b.append(100)`. Print `original` and explain why it changed.

Step-by-Step: Build It

One assemble: a tiny marks register that creates, updates, adds, removes, loops, and ranks — the same skills the lab menu will use.

```
students = [["Priya", 88], ["Ravi", 76]]

students[1][1] = 91
students.append(["Dev", 95])

removed = students.pop(0)
print("Removed:", removed[0])

for name, mark in students:
    print(f"{name}: {mark}")

marks_only = []
for name, mark in students:
    marks_only.append(mark)
```

```
ranked = sorted(marks_only, reverse=True)
print("Ranked marks:", ranked)
print("Live rows still:", students)
```

Expected Output:

```
Removed: Priya
Ravi: 91
Dev: 95
Ranked marks: [95, 91]
Live rows still: [['Ravi', 91], ['Dev', 95]]
```

`sorted(...)` ranks without destroying the live rows. `students.sort(...)` would rewrite the register itself.

Practice Exercises

Attempt each problem before reading the model answer.

1. From expenses `[120, 80, 450, 60]`, print only amounts above ₹100 using a `for` loop (not a comprehension).

Model answer:

```
expenses = [120, 80, 450, 60]
for amount in expenses:
    if amount > 100:
        print(amount)
```

Expected Output:

```
120
450
```

2. Start with `names = ["Ananya", "Ravi", "Priya"]`. Change index `1` to `"Ravi Kumar"`, then `insert(0, "Mentor")`. Print the list.

Model answer:

```
names = ["Ananya", "Ravi", "Priya"]
names[1] = "Ravi Kumar"
names.insert(0, "Mentor")
print(names)
```

Expected Output:

```
['Mentor', 'Ananya', 'Ravi Kumar', 'Priya']
```

3. From `tasks = ["Login", "API", "Deploy", "Docs"]`, cancel the last task with `pop()`, then `remove("API")`. Print the cancelled name and remaining list.

Model answer:

```
tasks = ["Login", "API", "Deploy", "Docs"]
cancelled = tasks.pop()
tasks.remove("API")
print(cancelled)
print(tasks)
```

Expected Output:

```
Docs
['Login', 'Deploy']
```

4. Given `original = [67, 91, 88]`, create a descending sorted **copy**. Prove `original` is unchanged. Then demonstrate the alias bug with `b = original` and `b.append(100)`.

Hint: `sorted` vs `=`.

Model answer:

```
original = [67, 91, 88]
ranked = sorted(original, reverse=True)
print(ranked)
print(original)

b = original
b.append(100)
print("after alias append:", original)
```

Expected Output:

```
[91, 88, 67]
[67, 91, 88]
after alias append: [67, 91, 88, 100]
```

5. Nested order lines `[["Tea", 2, 15], ["Samosa", 1, 25]]`: print each line total and the grand total.

Model answer:

```
order = [["Tea", 2, 15], ["Samosa", 1, 25]]
grand = 0
for name, qty, price in order:
    line = qty * price
    grand += line
    print(f"{name}: ₹{line}")
print(f"Grand: ₹{grand}")
```

Expected Output:

```
Tea: ₹30
Samosa: ₹25
Grand: ₹55
```

6. Pair `["Priya", "Dev"]` with `[88, 95]` using `zip` and print each as `Name: mark%`.

Model answer:

```
for name, mark in zip(["Priya", "Dev"], [88, 95]):
    print(f"{name}: {mark}%")
```

Expected Output:

```
Priya: 88%
Dev: 95%
```

Lab

Combine create, update, add, delete, loop, and a safe empty-list path. No `def`, no files.

Problem

A canteen keeps today's cart as a list of `[item, price]` rows. Staff add lines, drop a cancelled line, and print a total — without crashing when the cart is empty.

What to Build

A short script that seeds two rows, updates, adds, removes, lists, and totals.

Requirements

- Start with two rows, e.g. `[["Chai", 20], ["Samosa", 25]]`
- Update one price
- `append` a third item
- `pop` one row and print `Removed: <name>`
- Loop what remains as `name: ₹price`
- Print the total of remaining prices
- If you clear the cart, printing the total must not crash — print a safe message instead

Expected Behavior

After delete, listed items are only what remains. An empty cart prints a message, not a traceback.

Sample Run

```
Removed: Chai
Samosa: ₹25
Vada: ₹15
Total: ₹40
```

(Your seed data may differ; the **shape** of the output should match.)

Constraints

Top-level statements only — no `def`, no type hints, no files. No dictionary as the main store.

Bonus

Keep a backup with `.copy()` before you `pop`, then print both lists to show they are independent (flat rows of numbers/strings).

Mini-Project (Assignment): Student Marks Manager CLI

Why this matters: A class register is a list: add marks, list them, average, find toppers. Empty-list safety is the same bug every dashboard hits on day one.

Lab kit: 15_Lab_Assets/10_marks_manager/

File	What you do
`starter/marks_manager.py`	Build the `while True` menu. **No `def` required.**
`broken/marks_manager.py`	Empty average / toppers bugs — fix so empty list prints a safe message
`solution/`	Do **not** copy this

This is **not** the Step-by-Step assemble from class. Same list tools, full menu ticket.

Objective

Store marks in a list and drive add, list, average, and toppers from a menu. An empty list must not crash.

Story / Scenario

A class mentor types marks during lab. They need the average and everyone ≥ 90 — or a clear “no marks yet”, never a traceback.

Requirements

Piece	Detail
Data	`marks = []` or nested `[name, mark]` rows — either is fine
Menu	**1** Add mark (accept only **0–100**) · **2** List · **3** Average · **4** Toppers (**≥ 90**) · **5** Quit
Average	Uses the **current** list; empty list → safe message
Toppers	Collect with a normal `for` loop
Style	Top-level statements only — no `def`, no type hints

Expected Behavior

- Marks outside 0–100 are rejected
- Empty-list average prints a message
- Toppers lists everyone with mark ≥ 90
- Quit exits cleanly

Sample Run

```
=== Student Marks Manager ===
1. Add student mark
2. View all marks
3. Show class average
4. List toppers (>=90)
5. Exit
Choice: 1
Name: Priya
Mark: 88
Choice: 1
Name: Dev
Mark: 95
Choice: 2
Priya: 88
Dev: 95
Choice: 3
Class average: 91.50
Choice: 4
Dev: 95
Choice: 5
```

Bonus Challenges

- Show the name(s) with the single highest mark (ties OK)
- Reject non-numeric mark input with a clear message
- **Later — Functions week:** wrap add / average / toppers in `def`

Done when

- [ ] Menu options 1–5 work on the same list
- [ ] Marks outside 0–100 are rejected
- [ ] Empty-list average prints a safe message (no crash)
- [ ] Toppers lists everyone with mark ≥ 90
- [ ] No `def` / type hints on the core path

Interview Q&A

Q1. Why does indexing start at 0?
Python (and most languages) use zero-based indexing. Spreadsheet “row 1” is a human numbering convention — different rule.

Q2. `sort()` vs `sorted()`?
`sort()` changes the list in place and returns `None`. `sorted()` returns a **new** list; the original stays as it was.

Q3. Is `[1, 2] + [3]` the same as `extend`?
`+` creates a **new** list. `extend` mutates the original and returns `None`.

Q4. Why is modifying a list while iterating risky?
Items can be skipped. Prefer a new list, or iterate over a copy (`lst[:]`).

Q5. `b = a` vs `a.copy()`?
`b = a` aliases the same object. `.copy()` / `list(a)` / `a[:]` make a shallow copy of the outer list. Nested lists may still be shared.

Q6. Time complexity of `x in my_list`?

O(n) linear search. Sets are typically O(1) average for membership (next module).

Q7. What is a list comprehension (preview only)?

A compact "build a new list from a loop + optional filter," e.g. `[a for a in expenses if a > 500]`. Prefer a normal `for` until that pattern feels natural — not required for tonight's lab.

Cheat Sheet

```
[]                      → create a list
lst[i] / lst[a:b]       → read / slice (stop excluded)
len(lst)                → length
lst[i] = x              → replace
lst.append(x)           → add one at end
lst.extend(other)       → add many, flat
lst.insert(i, x)        → add at position
lst.remove(x)           → delete first match (ValueError if missing)
lst.pop() / lst.pop(i) → delete and return
lst.clear()             → empty
lst.count(x) / .index(x)
lst.sort()              → in place (returns None)
sorted(lst)             → new list
zip(a, b)               → pair two lists
b = a                   → alias (same object)
lst.copy()              → shallow copy (nested rows may be shared)
```

Order matters. Duplicates are allowed. Lists are mutable.